I.2 PAUSE TOKENS DO NOT ADD SEQUENTIAL COMPUTE

Comparing pause tokens with adding layers or attention heads Recall that <pause> tokens are added as a part of the input prompt, where each token is processed in parallel. Thus, <pause> tokens do not add extra serial computations. If there are sufficiently many parallel threads available, the wall clock overhead from pause tokens would be a minimal percentage of the time required for standard inference. However, in contrast, increasing the number of transformer layers increases the length of sequential operations, causing the wall-clock time to increase proportionally to the fraction of layers added. Note that adding attention heads should have a similar effect as adding pause tokens, as they too introduce parallel, not sequential operations.

Comparison with Chain-of-Thought (CoT) prompting Recall that CoT involves auto-regressively decoding a long sequence of tokens involved in the model's reasoning. This requires a significant wall clock time cost, proportional to $O(pl)$, if p is the number of intermediate reasoning tokens and l is the number of layers. In stark contrast, pause tokens do not add extra wall-clock time. Furthermore, CoT prompting offers little flexibility in how large p can be. Pause-inference on the other hand, offers a more direct way for manipulating the number of pause-tokens (even if, in its current version, this adaptivity is not robust beyond a point).

I.2.1 SUPPORTING LEMMAS FOR ESTIMATING FLOPS EFFICIENCY

In Lemma I.1, we present the facts about FLOPS required for basic vector and matrix calculations. Subsequently, in Lemma I.2, we compute the overall FLOPS required for an end-to-end Transformer computation. Finally, in Theorem I.3 and I.4, we derive how different kinds of parameter expansions in the model compare to adding pause tokens, in terms of FLOPS efficiency. Specifically, Theorem I.3 establishes the number of layers one needs to add to a model to realize the same number of FLOPS as adding p pause tokens. Theorem I.4 presents a similar result for adding attention heads.

Lemma I.1. ***(FLOPS for vector and matrix calculations)*** *The number of flops required to compute:*

1. *the dot product* $\mathbf{v}_1 \cdot \mathbf{v}_2$ *where* $\mathbf{v}_1, \mathbf{v}_2 \in \mathbb{R}^d$ *is* $O(d)$.
2. *the matrix multiplication* $M_1 \cdot M_2$*, where* $M_1 \in \mathbb{R}^{a\times b}$ *and* $M_2 \in \mathbb{R}^{b\times c}$*, the FLOPS is* $O(a \times b \times c)$.

Proof. For a dot product between $\mathbf{v}_1$ and $\mathbf{v}_2$, both of dimension d, the total number of FLOPS is given by the sum of multiplications and additions required. Specifically, it involves d multiplications and $d-1$ additions, totaling to $2d-1$ FLOPS. For the ease of calculation, we approximate this as $2d$.

For a matrix multiplication of $M_1 \in \mathbb{R}^{a\times b}$ and $M_2 \in \mathbb{R}^{b\times c}$, each element of the resulting matrix is computed by taking the dot product of a row from M_1 and a column from M_2, which requires $2b$ FLOPS . Since there are $a \times c$ such dot products to compute for the entire matrix multiplication, the total FLOPS amount to $(2b) \times a \times c$. However, for simplicity, if we only consider the multiplicative operations, the FLOPS reduce to $a \times b \times c$. □

Lemma I.2. ***(FLOPS for one end-to-end Transformer computation)*** *Consider an* l *layered decoder only language model, where we denote input embedding dimension as* h*, number of attention heads as* a *and per-attention-head embedding dimension as* d*. We assume feed-forward hidden dimension to be* $4h$ *and finally let* n *denote the length of input sequence. Then the total FLOPS are given as:*

$$F(n, h, d, a, l) = (4nadh + 2an^2d + 8nh^2)l \tag{9}$$

We note that standard Transformer implementations assume $d = h/a$, i.e. the per-attention-head embedding dimension decreases as the number of attention heads are increased. However, we treat these are three independent hyperparameters for greater flexibility in our analysis.

Proof. Let us consider the various per-layer operations in a decoder-only model step-by-step and count their FLOPS :

1. **q**, **k**, **v** vector computation: Given input token $x \in \mathbb{R}^h$, for the query vector computation, we have $Q^j = W_q^j x \forall j \in [1, a]$, where $W_q^j \in \mathbb{R}^{d\times h}$. The same extends for key and value vector computations. Thus total FLOPS required is $3nadh$.

2. Self-attention: Given $Q \in \mathbb{R}^{n \times d}$ and $K \in \mathbb{R}^{n \times d}$, QK^T incurs $n^2 d$ flops. The obtained $\alpha = softmax(\frac{QK^T}{\sqrt{d}}) \in \mathbb{R}^{n \times n}$ is multiplied by $V \in \mathbb{R}^{n \times d}$, costing another $n^2 d$ flops. Note that for simplicity, we ignore that FLOPS from softmax or the division by $\sqrt{d}$ operation as they are negligible. Thus, the total FLOPS = $a[n^2 d + n^2 d] = 2an^2 d$.

3. Combining multi-head-attention: The MHA projection matrix concatenates all the outputs from individual attention heads above and projects them to output of dimension h. For simplicity, we ignore the FLOPS from the skip connection as it adds only a relatively minimal number. Thus total FLOPS = $n \times h \times ad = nadh$.

4. Feed-forward network: This adds another $8nh^2$ FLOPS . Again, for simplicity we ignore the FLOPS from the skip connection.

Combining the FLOPS from each of the sub-parts above, we have:

$$FLOPS = (4nadh + 2an^2 d + 8nh^2)l \tag{10}$$

□

Theorem I.3. ***(FLOPS for adding k layers vs. p pause tokens)*** *Consider a l layer decoder only model, with h denoting the input embedding dimension and d denoting the per attention head embedding dimension. Let n be the length of initial prompt. Then, under the assumption that hidden embedding dimensions are much larger then the prompt sequence length and the appended pause tokens i.e. $d, h \gg n, p$; the additional FLOPS from p pause tokens is less than that from k additional layers of transformer if $n > pl/k$.*

Proof. From Lemma I.2 we have that increase in FLOPS due to k additional transformer layer is give by:

$$\begin{aligned} F_{\Delta l=k} &= F(n, h, a, l + k) - F(n, h, a, l) \\ F_{\Delta l=k} &= (4nadh + 2an^2 d + 8nh^2)k \end{aligned} \tag{11}$$

Similarly, increase in FLOPS due to p pause tokens is given by:

$$\begin{aligned} F_{\Delta n=p} &= F(n + p, h, a, l) - F(n, h, a, l) \\ F_{\Delta n=p} &= (4padh + 2ad((n + p)^2 - n^2) + 8ph^2)l \\ F_{\Delta n=p} &= (4adh + 2ad(2n + p) + 8h^2)pl \end{aligned} \tag{12}$$

Now,

$$\begin{aligned} F_{\Delta n=p} &< F_{\Delta l=k} \\ \implies (2adh + ad(2n + p) + 4h^2)pl &< (2adh + and + 4h^2)nk \\ \implies n &> \frac{(2adh + ad(2n + p) + 4h^2)pl}{(2adh + and + 4h^2)k} \\ \implies n &> \frac{pl}{k} \qquad \text{[assuming } h, d \gg n, p\text{]} \end{aligned} \tag{13}$$

□

Next, we derive how many attention heads can be added, to be FLOP-equivalent to adding p pause tokens. Note that in the standard transformer implementation, increasing the attention heads decreases the per attention head embedding dimension (i.e. $d = h/a$). In contrast, adding pause tokens, increases the number of attention computations while keeping the per-attention-head dimension fixed. Thus for a fair comparison, we consider a setting where we increase the number of attention heads, while keeping the per-attention-head dimension fixed. Specifically, we consider the case where per attention head embedding dimension is fixed to be the same of input embedding (i.e. $d = h$).

Theorem I.4. ***(FLOPS for adding k attention heads vs p pause tokens)*** *Consider a decoder only language model, with the per-attention-head embedding dimension d, fixed to be same as the input embedding dimension h. Let n be the length of initial prompt. Then under the assumption that hidden embedding dimension is much larger then the prompt sequence length and the appended pause tokens i.e. $d, h \gg n, p$; the additional FLOPS from p pause tokens is less than that from k additional attention head, if $n > \frac{(a+2)p}{k}$.*

Proof. From Lemma I.2, we have:

$$F(n, h, d, a, l) = (4anh^2 + 2an^2h + 8nh^2)l, \text{ where } d = h. \tag{14}$$

Now, increase in FLOPS due to k additional attention head is given by:

$$\begin{aligned} F_{\Delta a=k} &= F(n, h, a+k, l) - F(n, h, a, l) \\ F_{\Delta a=k} &= (4knh^2 + 2kn^2h)l \end{aligned} \tag{15}$$

Similarly, increase in FLOPS due to p pause tokens is given by:

$$\begin{aligned} F_{\Delta n=p} &= F(n+P, h, a, l) - F(n, h, a, l) \\ F_{\Delta n=p} &= (4aph^2 + 2ah((n+p)^2 - n^2) + 8ph^2)l \\ F_{\Delta n=p} &= (4ah + 2a(2n+p) + 8h)phl \end{aligned} \tag{16}$$

Therefore, we have, for:

$$\begin{aligned} F_{\Delta n=p} &< F_{\Delta a=1} \\ \implies (4ah + 4an + 2ap + 8h)hpl &< (4nh^2 + 2n^2h)lk \\ \implies ((2a+4)h + 2an + ap)p &< (2h+n)kn \\ \implies n &> \frac{(2a+4)h + 2an + ap}{(2h+n)k} \cdot p \\ \implies n &> \frac{(a+2)p}{k}, \qquad \text{[assuming } h, d \gg n, p] \end{aligned} \tag{17}$$

□

J THEORETICAL INTUITION

This section formalizes a broad class of problems where appending pause tokens during inference can enhance expressivity and thus be helpful. Our formalization identifies two core insights:

1. *Pause tokens can be critical to solve tasks that require a large number of independent parallel computations that exceed the number of input tokens.* For example, consider a task where the input is a sequence of L numbers $v_1, v_2, \ldots, v_L$, and the target is a polynomial of the form $(v_1 + v_2) \cdot (v_1 + v_3) \ldots (v_5 + v_2)$. If the number of addition operations required (N) scales much larger than the total number of input tokens (L), (and so $N = \omega(L)$) we argue that (under some natural capacity constraints), standard inference fails as it is bottlenecked in terms of its "implementational capacity": it can conduct only $O(L)$ operations in parallel. Pause-inference however is relieved of this bottleneck.
2. *The attention-feedforward block in any layer has "untapped" representational capacity — that is independent of the input length — which pause-inference taps into.* Specifically, note that the attention-feedforward block can implement many different operations, one for each intermediate vector it generates at each positional index. But crucially, the number of possible such operations (say, K) scales with the parameter count of the block. This quantity is independent of — and in practice, is much larger than — the input sequence length. Unfortunately, standard inference can only help realize at most L such operations (where $L \ll K$), while pause-inference can tap into K different such operations.